

BỘ GIÁO DỤC VÀ ĐÀO TẠO | CHƯƠNG TRÌNH CAO HỌC /
THẠC SĨ

BÁO CÁO KỸ THUẬT & TÀI LIỆU HƯỚNG DẪN DỰ ÁN

HƯỚNG DẪN THỰC HIỆN PROJECT MÔN AI AGENT

ĐỀ TÀI SỐ 2: TRỢ LÝ TUÂN THỦ & PHÁP LÝ DOANH NGHIỆP

(Legal & Compliance AI Assistant)

Kỹ thuật RAG nâng cao: Corrective RAG (CRAG)

Chuẩn bị dữ liệu – Thiết kế Database – Cài đặt Agent – Thực nghiệm Benchmark – Đánh giá

Môn học:	Kỹ thuật Xây dựng AI Agent (Advanced AI Agents)
Trọng tâm nghiên cứu:	Cơ chế tự hiệu chỉnh Retrieval trước khi sinh câu trả lời
Kiến trúc điều phối:	LangGraph + Multi-Provider LLM (Groq, OpenAI, Open-Router, Ollama) + Chroma
Giao diện & Backend:	Next.js (TypeScript) + FastAPI (RESTful API)
Thời gian hoàn thành:	Học kỳ 2 – Năm học 2025 – 2026

Tóm tắt Dự án

Tài liệu này là Báo cáo Kỹ thuật và Hướng dẫn Triển khai Toàn diện cho **Đề tài số 2: Trợ lý Tuân thủ & Pháp lý Doanh nghiệp (Legal & Compliance AI Assistant)** thuộc chương trình môn học AI Agent nâng cao cấp Thạc sĩ. Hệ thống giải quyết căn nguyên của vấn đề ảo giác (*hallucination*) trong các hệ thống hỏi đáp văn bản quy phạm pháp luật thông qua kỹ thuật **Corrective RAG (CRAG)**.

Khác với các kiến trúc RAG truyền thống vốn mặc định tin tưởng tuyệt đối vào tài liệu truy hồi, hệ thống Legal CRAG V3 triển khai một cơ chế đánh giá độc lập (*Retrieval Evaluator*) sử dụng mô hình Cross-Encoder để phân loại chất lượng bằng chứng thành ba nhánh rõ điều kiện:

- Nhánh Correct:** Kích hoạt cơ chế tinh lọc tri thức (*Knowledge Refinement*) để bóc tách các điều khoản (*legal strips*) chính xác, loại bỏ phần thừa trước khi chuyển tới bộ sinh.
- Nhánh Incorrect:** Nhận diện sự thiếu hụt bằng chứng trong cơ sở tri thức nội bộ, kích hoạt viết lại truy vấn (*Query Rewrite*) và thực hiện tìm kiếm có kiểm soát (*Controlled Web Search*) trên các cổng thông tin pháp luật chính thống của nhà nước.
- Nhánh Ambiguous:** Khắc phục đứt điểm hiện tượng lặp lại máy móc của các phiên bản trước bằng cách kết hợp đồng thời việc tinh lọc bằng chứng nội bộ với việc tìm kiếm mở rộng nguồn ngoài, sau đó hợp nhất (*Evidence Merging*) có ưu tiên độ tin cậy.

Toàn bộ hệ thống được xây dựng trên nền tảng **LangGraph**, kiến trúc mô hình ngôn ngữ lớn linh hoạt hỗ trợ đa nhà cung cấp (**Multi-Provider LLM: Groq, OpenAI GPT, OpenRouter, Ollama**), kỹ thuật truy hồi lai **Hybrid Retrieval (Dense BGE-M3 + Lexical BM25 + RRF)**, cơ sở dữ liệu quan hệ **SQLite**, bộ kiểm định trích dẫn xác định **Citation Validator**, giao diện người dùng **Next.js (TypeScript)** và cơ chế quản lý **Session Memory** tách biệt theo cặp ản danh `client_id/session_id`.

Từ khóa: *Corrective RAG, AI Agent, LangGraph, Multi-Provider LLM, Groq, OpenAI, OpenRouter, Ollama, Hybrid Retrieval, Citation Validation, Session Memory.*

Contents

Tóm tắt Dự án	i
1 Xác định Phạm vi và Mục tiêu Đề tài	1
1.1 Tên Đề tài và Mục tiêu Nghiên cứu	1
1.2 Phạm vi Khuyến nghị	1
1.3 Hệ thống Câu hỏi Mẫu cần Xử lý	2
1.4 Câu hỏi Nghiên cứu & Thực nghiệm	2
2 Kiến trúc Tổng thể và Luồng Hoạt động CRAG	3
2.1 Kiến trúc 3 Layer Tách Biệt	3
2.2 Luồng Hoạt động CRAG Cốt lõi (3 Nhánh)	3
2.3 Stack Công nghệ Đề xuất	5
3 Quy trình Xây dựng Hệ thống (18 Bước Hiện thực)	7
3.1 BƯỚC 1: Chuẩn bị Môi trường & Cấu hình Trung tâm	7
3.2 BƯỚC 2: Tổ chức Thư mục Dự án	8
3.3 BƯỚC 3: Chuẩn bị Corpus Pháp luật	9
3.4 BƯỚC 4 & 5: Thiết kế và Tạo Cơ sở Dữ liệu Pháp lý	9
3.5 BƯỚC 6: Cấu hình LLM Đa Provider, Embedding và Reranker	11
3.6 BƯỚC 7: Định nghĩa Agent State	12
3.7 BƯỚC 8: Ingestion Pipeline & Hybrid Retrieval	13
3.8 BƯỚC 9: Xây dựng các Tools	13
3.8.1 Reciprocal Rank Fusion (RRF)	13
3.8.2 Controlled Web Search Tool	13
3.9 BƯỚC 10: Xây dựng CRAG Retrieval Evaluator	14
3.10 BƯỚC 11 – 14: Tinh chế Tri thức, Viết lại Truy vấn & Hợp nhất Bằng chứng	14
3.11 BƯỚC 15: Kiểm định Trích dẫn (Citation Validator)	15
3.12 BƯỚC 16: Prompt Sinh câu trả lời có Cấu trúc	15
3.13 BƯỚC 17: Xây dựng LangGraph Hoàn chỉnh	16
3.14 BƯỚC 18: Kịch bản Kiểm thử Bắt buộc khi Demo	17
4 Thiết kế Thực nghiệm & Đánh giá Hệ thống	19
4.1 Thiết kế Hệ thống Baseline và Nghiên cứu Ablation	19
4.2 Thiết kế Tập Calibration và Held-Out Test Set	20
4.3 Hệ thống Chỉ số Đánh giá Toàn diện	21
4.3.1 Chỉ số Đánh giá Truy hồi (Retrieval Metrics)	21
4.3.2 Chỉ số Đánh giá Bộ điều hướng & Fallback (Routing & Fallback Metrics)	21

4.3.3	Chỉ số Đánh giá Hệ Sinh & Độ chuẩn xác Pháp lý (Legal Metrics) . . .	21
4.4	Thí nghiệm Lựa chọn Cặp Ngưỡng Relevance (T_{low}, T_{high})	21
5	Mở rộng: Next.js Frontend & Quản lý Session Memory	23
5.1	Kiến trúc Tích hợp Next.js + FastAPI + Memory	23
5.2	Phân cấp Ba Lớp Memory	23
5.3	Thiết kế Database cho Client, Session và Memory	24
5.4	Kiểm soát Semantic Memory qua Danh mục Cho phép (Allow-List)	25
5.5	Prompt Trả lời Phân tách Bối cảnh và Căn cứ Pháp lý	25
5.6	Kịch bản Demo Tái sử dụng Ngữ cảnh trên Trình duyệt	26
5.7	Bảo mật và Tiêu chuẩn Cách ly Dữ liệu (Isolation)	26
5.8	Đánh giá Thử nghiệm A/B Memory	26
6	Kế hoạch Thực hiện, Checklist & Hướng dẫn Báo cáo	28
6.1	Checklist CRAG Core Trước khi Triển khai Mở rộng	28
6.2	Lộ trình 17 Giai đoạn Triển khai Dự án	28
6.3	Đề cương Chi tiết 8 Chương của Báo cáo Kỹ thuật	28
7	Phụ lục & Tài liệu Tham khảo	31
7.1	PHỤ LỤC A: Danh mục File Mã nguồn Chuẩn hóa	31
7.2	PHỤ LỤC B: Gợi ý Phân công Công việc Nhóm (5 Thành viên)	31
7.3	PHỤ LỤC C: Tiêu chí Tự Kiểm tra Chất lượng Trước khi Nộp	31
7.4	PHỤ LỤC D: So sánh Chi tiết các Thay đổi giữa Bản cũ và Bản V3	31
	Tài liệu Tham khảo	31

List of Figures

2.1	Kiến trúc 3 Layer của Legal CRAG Agent V3.	3
2.2	Luồng xử lý 3 nhánh Correct / Ambiguous / Incorrect theo nguyên lý CRAG. . . .	4
5.1	Kiến trúc tách biệt giữa ngữ cảnh người dùng và căn cứ pháp lý.	23
6.1	Thứ tự thực hiện chuẩn mực: Ưu tiên dữ liệu, đo lường và lỗi CRAG trước khi làm UI.	30

List of Tables

2.1	Phân định chức năng và vai trò của từng Layer trong hệ thống	5
2.2	Chi tiết Stack công nghệ triển khai hệ thống Legal CRAG V3	6
3.1	Quy mô corpus và các tập dữ liệu thực nghiệm	9
3.2	5 Kịch bản kiểm thử bắt buộc cần demo để minh chứng hoạt động của CRAG . . .	18
4.1	Ma trận các hệ thống Baseline thực nghiệm (S0 – S4)	19
4.2	Phân bổ các nhóm câu hỏi trong Held-out Test Set (40 câu)	20
4.3	Bảng khung thực nghiệm quét lưới lựa chọn ngưỡng (Grid Search Calibration) . .	22
5.1	Phân tầng 3 cấp độ Memory trong hệ thống	23
5.2	Kịch bản minh chứng khả năng duy trì ngữ cảnh trên cùng trình duyệt	26
6.1	Kế hoạch thực hiện dự án theo 17 giai đoạn chuẩn hóa	29
7.1	Danh mục các module mã nguồn trong hệ thống Legal CRAG V3	32
7.2	Mô hình phân công trách nhiệm trong nhóm thực hiện đề tài	33
7.3	Bảng đối chiếu 15 điểm cải tiến then chốt của Phiên bản V3	34

Xác định Phạm vi và Mục tiêu Đề tài

Nguyên tắc cốt lõi của Bản V3

CRAG Core phải chạy và được đánh giá trước. Next.js frontend và Semantic Memory là phần mở rộng kiến trúc; memory được liên kết chặt chẽ theo cặp ẩn danh `client_id / session_id` và **tuyệt đối không được dùng như một nguồn căn cứ pháp lý.**

1.1 Tên Đề tài và Mục tiêu Nghiên cứu

Tên đề tài đề xuất: *AI Agent hỗ trợ tra cứu pháp lý doanh nghiệp Việt Nam với cơ chế tự hiệu chỉnh truy hồi dựa trên Corrective RAG (CRAG).*

Đề tài tập trung xây dựng một hệ thống AI Agent chuyên sâu có khả năng:

1. Tra cứu văn bản quy phạm pháp luật với độ chính xác cao.
2. Tự động đánh giá chất lượng bằng chứng (*evidence evaluation*) được truy hồi từ corpus nội bộ thông qua mô hình chấm điểm độc lập.
3. Tự động quyết định và kích hoạt các hành động hiệu chỉnh thích ứng (*Corrective Actions*): tinh lọc tri thức nội bộ (*Knowledge Refinement*), viết lại truy vấn (*Query Rewrite*) và tìm kiếm nguồn mở rộng có kiểm soát (*Controlled Web Search*) khi dữ liệu nội bộ không đáp ứng.
4. Chỉ sinh câu trả lời khi có bằng chứng pháp lý rõ ràng được đối chiếu thông qua cơ chế kiểm định trích dẫn (*Citation Validation*).

Mục tiêu trọng tâm: Không phải xây dựng một chatbot trả lời chung chung mọi câu hỏi pháp luật, mà là thiết kế, hiện thực và đánh giá một **pipeline Corrective RAG có thể quan sát, đo lường được và giải thích được.**

1.2 Phạm vi Khuyến nghị

Để đảm bảo tính khả thi trong việc xây dựng ground-truth thủ công đạt chuẩn học thuật mà vẫn giữ được tính đa dạng của bài toán pháp lý phức tạp, phạm vi hệ sinh thái tri thức được giới hạn trong hai lĩnh vực trọng tâm:

- **Đăng ký doanh nghiệp:** Điều kiện thành lập, hồ sơ, trình tự, thủ tục hành chính, cơ quan tiếp nhận và thẩm quyền xử lý, tra cứu tình trạng hiệu lực văn bản (Luật Doanh nghiệp 2020 và các Nghị định hướng dẫn liên quan).
- **Pháp luật lao động:** Hợp đồng lao động, thời hạn, quyền và nghĩa vụ các bên, thủ tục chấm dứt hợp đồng, trách nhiệm pháp lý của người sử dụng lao động (Bộ luật Lao động 2019 và văn

bản liên quan).

- **Giới hạn phạm vi:** Không mở rộng sang toàn bộ hệ thống thuế, bảo hiểm xã hội, an toàn vệ sinh lao động trong giai đoạn đầu nhằm đảm bảo chất lượng kiểm tra ground-truth.

1.3 Hệ thống Câu hỏi Mẫu cần Xử lý

Hệ thống được thiết kế để vượt qua các tình huống kiểm thử đại diện:

1. **Tra cứu quy định hiện hành:** “Điều kiện thành lập công ty TNHH theo quy định hiện hành là gì?”
2. **Tra cứu trạng thái hiệu lực lịch sử/hiện tại:** “Văn bản số X còn hiệu lực tại ngày Y hay không?”
3. **Tra cứu chi tiết điều khoản:** “Quy định hiện hành về hợp đồng lao động xác định thời hạn bao gồm những nội dung bắt buộc nào?”
4. **Nhận biết thiếu hụt tri thức (Out-of-corpus):** Khi kho dữ liệu nội bộ chưa cập nhật một nghị định mới sửa đổi, Agent phải nhận diện được sự thiếu hụt bằng chứng và kích hoạt tìm kiếm nguồn mở rộng chính thống.
5. **Xung đột văn bản pháp lý:** Hai nguồn đưa ra quy định khác nhau; Agent phải phân tích, phát hiện xung đột và ưu tiên áp dụng văn bản có thứ bậc pháp lý cao hơn hoặc còn hiệu lực tại thời điểm tra cứu.
6. **Xử lý hội thoại có ngữ cảnh (Multi-turn Context):** Người dùng hỏi câu hỏi tiếp diễn có đại từ: “Vậy đối với trường hợp công ty tôi thì cần lưu ý gì?” – Agent sử dụng memory để giải tham chiếu thực thể nhưng vẫn bắt buộc truy hồi lại căn cứ pháp luật hiện hành.

1.4 Câu hỏi Nghiên cứu & Thực nghiệm

5 Câu hỏi Nghiên cứu Trọng tâm

- **RQ1:** Liệu Corrective RAG có giảm thiểu đáng kể lỗi ảo giác do truy hồi kém chất lượng so với Traditional RAG hay không?
- **RQ2:** Retrieval Evaluator với mô hình Cross-Encoder có phân loại chính xác ba nhánh Correct / Ambiguous / Incorrect hay không?
- **RQ3:** Cơ chế Hybrid Retrieval (BGE-M3 Dense + BM25 Lexical + Reranker) cải thiện Recall@K và MRR như thế nào so với Dense-only retrieval?
- **RQ4:** Sự cải thiện vượt trội của CRAG thực sự đến từ cơ chế hiệu chỉnh tri thức hay chỉ đơn thuần nhờ được bổ sung thêm nguồn Web Search bên ngoài?
- **RQ5:** Semantic Memory và Session History có hỗ trợ ngữ cảnh hội thoại nhiều lượt mà không làm giảm độ tin cậy (faithfulness) hay làm rò rỉ dữ liệu giữa các phiên (cross-client contamination)?

Kiến trúc Tổng thể và Luồng Hoạt động CRAG

2.1 Kiến trúc 3 Layer Tách Biệt

Kiến trúc phiên bản V3 được chuẩn hóa thành 3 tầng chức năng độc lập, phân định ranh giới rõ ràng giữa Giao diện, Xử lý tác tử và Cơ sở dữ liệu.

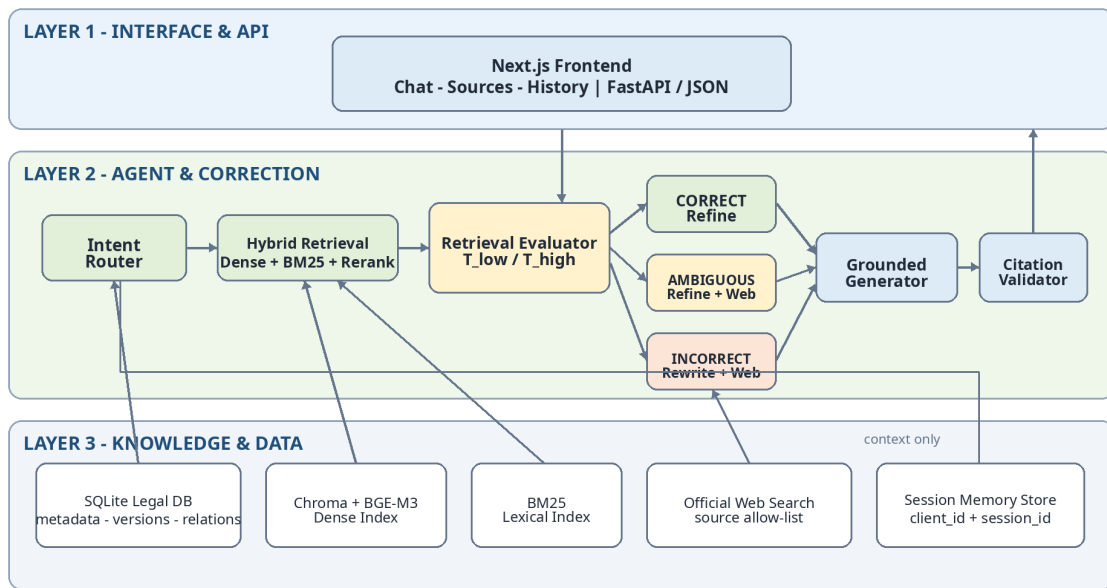


Figure 2.1: Kiến trúc 3 Layer của Legal CRAG Agent V3.

2.2 Luồng Hoạt động CRAG Cốt lõi (3 Nhánh)

Trọng tâm của kỹ thuật Corrective RAG là không bao giờ tin tưởng tuyệt đối vào kết quả truy hồi ban đầu. Thay vào đó, một bộ đánh giá (*Retrieval Evaluator*) sẽ phân loại tập tài liệu truy hồi thành một trong ba trạng thái:

- **Nhánh CORRECT (Tài liệu đạt chuẩn):** Điểm liên quan cao vượt ngưỡng T_{high} . Hệ thống kích hoạt *Knowledge Refinement* để bóc tách các điều khoản (legal strips) trực tiếp trả lời câu hỏi, loại bỏ nhiễu và chuyển sang bộ sinh câu trả lời.
- **Nhánh INCORRECT (Tài liệu không phù hợp/thiếu hụt):** Điểm liên quan tối đa nằm dưới ngưỡng T_{low} . Hệ thống xác định corpus nội bộ không có căn cứ, loại bỏ toàn bộ tài liệu rác, thực hiện *Query Rewrite* thành truy vấn tìm kiếm chuẩn mực và gọi công cụ *Controlled Web Search* qua các cổng thông tin pháp luật chính thống.
- **Nhánh AMBIGUOUS (Tài liệu không rõ ràng/chưa đủ):** Điểm liên quan nằm giữa khoảng $[T_{low}, T_{high}]$. Hệ thống vừa tinh chế tài liệu nội bộ, vừa viết lại truy vấn để tìm kiếm bổ sung.

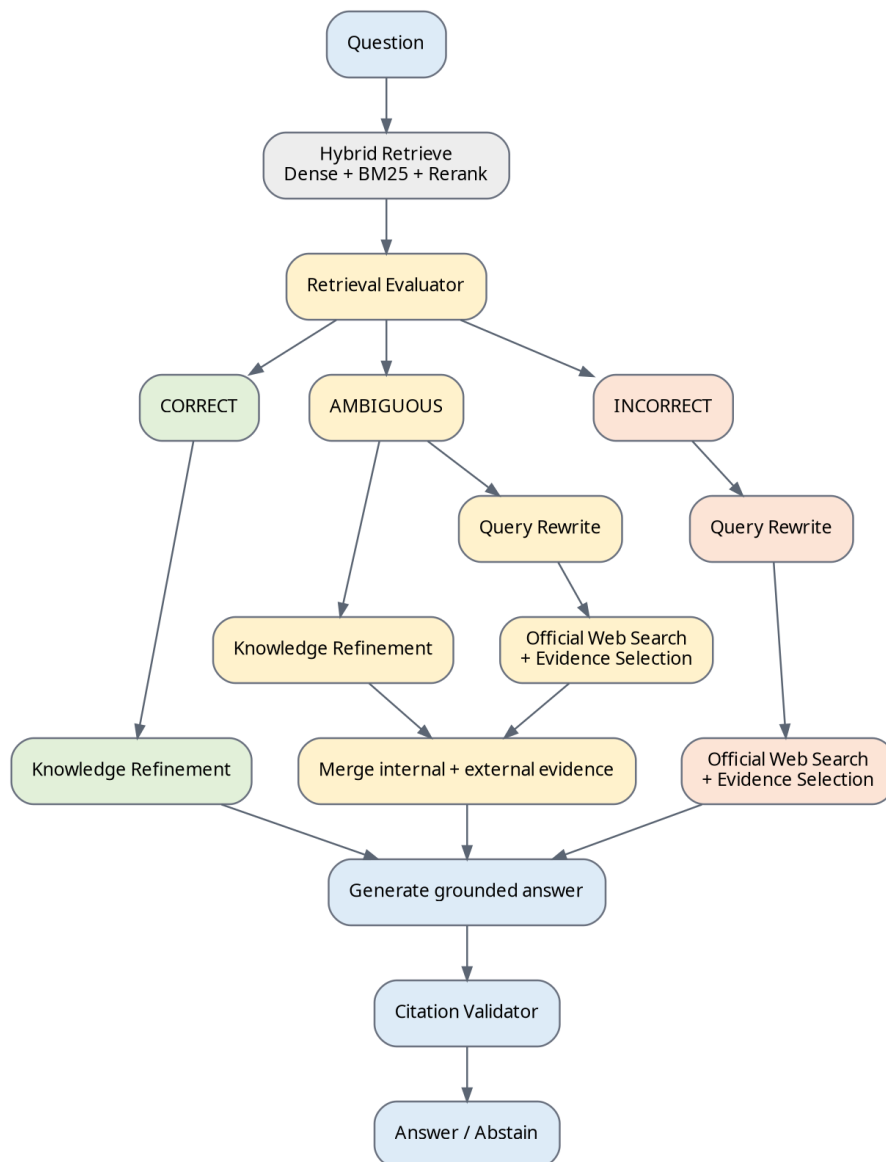


Figure 2.2: Luồng xử lý 3 nhánh Correct / Ambiguous / Incorrect theo nguyên lý CRAG.

Table 2.1: Phân định chức năng và vai trò của từng Layer trong hệ thống

Layer / Khối chức năng	Vai trò và Trách nhiệm chính
Layer 1 – Interface & API	Next.js + TypeScript phụ trách giao diện người dùng chuyên nghiệp; FastAPI cung cấp hạ tầng RESTful/JSON API kết nối trực tiếp với Agent, hỗ trợ hiển thị câu trả lời, bảng trích dẫn citation, danh mục source và quản lý phiên.
Layer 2 – Agent & Correction	LangGraph điều phối toàn bộ workflow: Router phân loại câu hỏi, Retrieval Evaluator chấm điểm, 3 nhánh xử lý CRAG cốt lõi, Generator và Citation Validator đảm bảo độ chính xác.
Layer 3 – Knowledge & Data	SQLite quản lý metadata pháp lý và lịch sử phiên, Chroma DB lưu trữ dense vector index, BM25 lưu lexical index, Reranker tính toán relevance, và công cụ Controlled Web Search với danh sách nguồn cho phép.

trên web, sau đó tiến hành *Evidence Merging* hợp nhất và khử trùng lặp trước khi sinh lời giải.

Điểm cải tiến then chốt so với các thiết kế cũ

Các phiên bản trước thường đưa nhánh *Ambiguous* quay lại vòng lặp truy hồi (*looping*) với biến `MAX_ITER`. Thiết kế V3 loại bỏ việc lặp lại máy móc: nhánh *Ambiguous* bắt buộc phải kết hợp tinh lọc tri thức nội bộ và mở rộng nguồn web có kiểm soát, giải quyết dứt điểm vấn đề thiếu bằng chứng mà không gây lặp vô hạn.

2.3 Stack Công nghệ Đề xuất

Table 2.2: Chi tiết Stack công nghệ triển khai hệ thống Legal CRAG V3

Thành phần	Công nghệ lựa chọn	Lý do và Ghi chú kỹ thuật
Ngôn ngữ nền tảng	Python 3.11+	Đảm bảo tính tương thích cao với LangGraph, PyTorch và transformers.
LLM Engine	Đa Provider (Groq, OpenAI, OpenRouter, Ollama)	Kiến trúc Factory linh hoạt: Groq (siêu tốc), OpenAI GPT-4o, OpenRouter và Ollama (local on-premise); cấu hình qua LLM_PROVIDER; cố định temperature=0.
Embedding Model	BAAI/BGE-M3	Embedding đa ngữ mạnh mẽ, xử lý tốt văn bản pháp luật tiếng Việt.
Dense Vector DB	ChromaDB	Nhẹ, nhúng trực tiếp, tối ưu cho quy mô 50–100 văn bản pháp lý.
Lexical Retrieval	rank-bm25 (BM25Okapi)	Bù đắp exact-match chính xác cho số hiệu văn bản, số Điều/Khoản.
Reranker	BAAI/bge-reranker-v2-m3	Cross-encoder tính relevance score chuẩn xác cho Retrieval Evaluator.
Agent Orchestrator	LangGraph	Điều phối workflow dạng đồ thị có trạng thái (<i>StateGraph</i>) và rẽ nhánh điều kiện.
Cơ sở dữ liệu	SQLite	Quản lý quan hệ văn bản, phiên bản điều luật, session logs và memory.
Web Search	Custom Engine + Allow-list	Chỉ thu thập từ các tên miền công quyền: vbp1.vn, chinhphu.vn...
Đánh giá chất lượng	RAGAS + Custom Metrics	Đo lường Faithfulness, Citation Accuracy và Legal Temporal Correctness.
Theo dõi	LangFuse	Ghi nhận toàn bộ vết thực thi, latency và chi phí token của từng node.
Frontend	Next.js + TypeScript	Giao diện tra cứu hiện đại, hiển thị bảng bằng chứng và lịch sử hội thoại.
API Gateway	FastAPI + Uvicorn	Cung cấp API chuẩn hóa REST/JSON kết nối UI và LangGraph Agent.

Quy trình Xây dựng Hệ thống (18 Bước Hiện thực)

3.1 BƯỚC 1: Chuẩn bị Môi trường & Cấu hình Trung tâm

Hệ thống yêu cầu cài đặt môi trường ảo riêng biệt để cố định thư viện trước khi chạy các phép đo kiểm chuẩn (*benchmarks*).

```

1 python -m venv .venv
2 source .venv/bin/activate # Trên Linux/macOS (.venv\Scripts\activate trên
   Windows)
3
4 pip install langgraph langchain langchain-chroma chromadb \
5         langchain-groq langchain-openai langchain-ollama \
6         rank-bm25 FlagEmbedding pypdf beautifulsoup4 pandas pydantic \
7         ragas datasets langfuse fastapi uvicorn[standard]

```

Listing 3.1: Khởi tạo môi trường ảo và cài đặt các phụ thuộc

Toàn bộ tham số hoạt động được gom về một module cấu hình thống nhất, tránh ghi cứng (*hardcoding*) rải rác:

```

1 # config.py
2 import os
3
4 # LLM Provider: "groq" | "openai" | "openrouter" | "ollama"
5 LLM_PROVIDER = os.getenv("LLM_PROVIDER", "groq")
6
7 # Model theo provider (ví dụ: Groq Llama 3.3 70B, OpenAI GPT-4o-mini, Ollama
   Qwen)
8 LLM_MODEL = os.getenv("LLM_MODEL", "llama-3.3-70b-versatile")
9 LLM_TEMPERATURE = 0.0
10
11 EMBEDDING_MODEL = "BAAI/bge-m3"
12 RERANKER_MODEL = "BAAI/bge-reranker-v2-m3"
13
14 TOP_K_DENSE = 20
15 TOP_K_BM25 = 20
16 TOP_K_RERANK = 5
17
18 # Ngưỡng khởi tạo (sẽ được tối ưu hóa trên Calibration Set)
19 T_LOW = 0.35
20 T_HIGH = 0.70
21 INTERNAL_STRIP_MIN = 0.40

```

Listing 3.2: File cấu hình trung tâm config.py

3.2 BƯỚC 2: Tổ chức Thư mục Dự án

Phiên bản V3 tuân thủ nguyên lý tách biệt trách nhiệm (*Separation of Concerns*):

```

1 legal_crag_agent/
2   data/
3     raw/                # File văn bản PDF/DOCX gốc
4     processed/          # File JSON đã parse cấu trúc pháp lý
5     corpus_manifest.json # Thông tin phiên bản và quy mô corpus
6   agent/
7     state.py            # Định nghĩa TypedDict AgentState
8     nodes.py            # Các hàm xử lý tại từng node
9     graph.py            # Đồ thị LangGraph và các conditional edges
10  retrieval/
11    dense.py             # Truy hỏi vector ChromaDB
12    bm25.py              # Truy hỏi từ khóa rank-bm25
13    fusion.py            # Thuật toán hợp nhất RRF
14    reranker.py          # Cross-encoder tính relevance score
15    refine.py            # Knowledge refinement chia nhỏ thành legal
16    strips
17  tools/
18    database.py          # Truy vấn metadata cơ sở dữ liệu SQLite
19    web_search.py        # Tìm kiếm mở rộng có kiểm soát tên miền
20  legal/
21    parser.py            # Tách văn bản theo Chương/Diều/Khoản/Diem
22    citations.py         # Kiểm định citation hợp lệ
23    temporal.py          # Xử lý và đối chiếu hiệu lực theo thời gian
24  memory/
25    store.py             # Quản lý Client, Session và Memory SQLite
26    extractor.py         # Trích xuất Semantic Memory theo allow-list
27  eval/
28    calibration_set.json  # 20 câu dùng để chọn ngưỡng T_low/T_high
29    test_set.json        # 40 câu held-out dùng đánh giá cuối
30    metrics.py           # Custom metrics: Action F1, Fallback, Citation
31    run_eval.py          # Script chạy benchmark tự động
32  api/
33    main.py              # FastAPI ứng dụng REST/JSON
34  frontend/
35    config.py            # Tham số hệ thống
36    llm.py               # Khởi tạo Ollama Chat và Embeddings
37    create_db.py         # Script tạo cơ sở dữ liệu app.db
38    ingest.py            # Pipeline nạp dữ liệu vào DB và Vector Index
39    main.py              # Giao diện dòng lệnh (CLI) test nội bộ

```

Listing 3.3: Cấu trúc cây thư mục của project Legal CRAG Agent

3.3 BƯỚC 3: Chuẩn bị Corpus Pháp luật

Corpus tập trung vào 50–100 văn bản chính thức với metadata đầy đủ, bảo đảm kiểm tra được tình trạng hiệu lực và quan hệ sửa đổi/bổ sung.

Table 3.1: Quy mô corpus và các tập dữ liệu thực nghiệm

Thành phần dữ liệu	Quy mô chuẩn hóa
Số lượng văn bản quy phạm pháp luật	50 – 100 văn bản (Luật, Nghị định, Thông tư)
Số lượng điều khoản (Provisions)	Khoảng 500 – 1.500 Điều / Khoản / Điểm
Số lượng đoạn trích ứng viên (Candidate chunks)	Khoảng 1.000 – 3.000 chunks có cấu trúc
Tập dữ liệu tinh chỉnh ngưỡng (Calibration Set)	20 câu hỏi có gán nhãn ground-truth
Tập kiểm thử độc lập (Held-out Test Set)	40 câu hỏi độc lập với Calibration Set

Legal-aware Chunking: Không dùng kỹ thuật cắt đoạn cố định (*fixed-size recursive chunking 700/100*) gây đứt gãy câu chữ pháp lý. Thay vào đó, văn bản được phân tách theo đúng cấu trúc thứ bậc:

Văn bản → Chương → Điều → Khoản → Điểm

Mỗi đoạn trích luôn lưu giữ đầy đủ đường dẫn ngược (*provenance*) về văn bản gốc.

3.4 BƯỚC 4 & 5: Thiết kế và Tạo Cơ sở Dữ liệu Pháp lý

Schema cơ sở dữ liệu V3 hỗ trợ mô hình hóa phiên bản điều luật theo thời gian và quan hệ sửa đổi phức tạp:

```
1 CREATE TABLE legal_documents (  
2     id TEXT PRIMARY KEY,  
3     document_number TEXT UNIQUE NOT NULL,  
4     title TEXT NOT NULL,  
5     document_type TEXT,  
6     issuing_authority TEXT,  
7     issued_at DATE,  
8     effective_from DATE,  
9     effective_to DATE,  
10    status TEXT,  
11    source_url TEXT NOT NULL,
```

```

12     retrieved_at TEXT
13 );
14
15 CREATE TABLE provisions (
16     id TEXT PRIMARY KEY,
17     document_id TEXT NOT NULL,
18     chapter TEXT,
19     article TEXT,
20     clause TEXT,
21     point TEXT,
22     heading TEXT,
23     text TEXT NOT NULL,
24     FOREIGN KEY(document_id) REFERENCES legal_documents(id)
25 );
26
27 CREATE TABLE provision_versions (
28     id INTEGER PRIMARY KEY AUTOINCREMENT,
29     provision_id TEXT NOT NULL,
30     valid_from DATE,
31     valid_to DATE,
32     text TEXT NOT NULL,
33     source_document_id TEXT,
34     FOREIGN KEY(provision_id) REFERENCES provisions(id)
35 );
36
37 CREATE TABLE legal_relations (
38     id INTEGER PRIMARY KEY AUTOINCREMENT,
39     source_document_id TEXT NOT NULL,
40     target_document_id TEXT NOT NULL,
41     source_provision_id TEXT,
42     target_provision_id TEXT,
43     relation_type TEXT NOT NULL, -- 'amends', 'supplements', 'replaces',
44     'repeals', 'guides'
45     note TEXT
46 );

```

Listing 3.4: Schema cơ sở dữ liệu pháp lý SQLite

```

1 import sqlite3
2
3 def init_db(db_path="app.db"):
4     with sqlite3.connect(db_path) as con:
5         con.execute("PRAGMA foreign_keys = ON")
6         con.executescript(open("schema.sql", encoding="utf-8").read())
7         con.execute("CREATE INDEX IF NOT EXISTS idx_doc_number ON
8             legal_documents(document_number)")
9         con.execute("CREATE INDEX IF NOT EXISTS idx_doc_status ON
10             legal_documents(status)")
11         con.execute("CREATE INDEX IF NOT EXISTS idx_provision_doc ON

```

```

        provisions(document_id)")
10     con.execute("CREATE INDEX IF NOT EXISTS idx_relation_target ON
        legal_relations(target_document_id)")
11     print("Database và các Index đã được ở khi ạo thành công.")

```

Listing 3.5: Script tạo cơ sở dữ liệu và chỉ mục create_db.py

3.5 BƯỚC 6: Cấu hình LLM Đa Provider, Embedding và Reranker

Hệ thống thiết kế tầng LLM theo mô hình kiến trúc cắm rút (*Pluggable Provider Pattern*). Ollama chạy local chỉ là một trong các nhà cung cấp; hệ thống có thể dễ dàng chuyển đổi hoặc mở rộng sang **Groq** (suy luận siêu tốc < 1s), **OpenAI** (GPT-4o, GPT-4o-mini), **OpenRouter** (truy cập hơn 100 mô hình mở/đóng) thông qua biến môi trường hoặc file cấu hình mà không phải sửa code Agent.

```

1  # llm.py
2  import os
3  from langchain_core.language_models.chat_models import BaseChatModel
4  from config import LLM_PROVIDER, LLM_MODEL, LLM_TEMPERATURE
5
6  def get_chat_model(provider: str = None, model: str = None) -> BaseChatModel:
7      p = (provider or LLM_PROVIDER).lower()
8      m = model or LLM_MODEL
9
10     if p == "groq":
11         from langchain_groq import ChatGroq
12         return ChatGroq(model=m, temperature=LLM_TEMPERATURE)
13     elif p == "openai":
14         from langchain_openai import ChatOpenAI
15         return ChatOpenAI(model=m, temperature=LLM_TEMPERATURE)
16     elif p == "openrouter":
17         from langchain_openai import ChatOpenAI
18         return ChatOpenAI(
19             model=m,
20             temperature=LLM_TEMPERATURE,
21             base_url="https://openrouter.ai/api/v1",
22             api_key=os.getenv("OPENROUTER_API_KEY")
23         )
24     elif p == "ollama":
25         from langchain_ollama import ChatOllama
26         return ChatOllama(model=m, temperature=LLM_TEMPERATURE)
27     else:
28         raise ValueError(f"Unsupported LLM provider: {p}")
29
30 llm = get_chat_model()
31
32 # Khởi tạo Embedding đa ngu BGE-M3
33 from langchain_ollama import OllamaEmbeddings
34 embeddings = OllamaEmbeddings(model="bge-m3")

```

```

35
36 # retrieval/reranker.py
37 from FlagEmbedding import FlagReranker
38 from math import exp
39 from config import RERANKER_MODEL
40
41 reranker = FlagReranker(RERANKER_MODEL, use_fp16=True)
42
43 def sigmoid(x: float) -> float:
44     return 1.0 / (1.0 + exp(-float(x)))
45
46 def score_pairs(query: str, docs: list[dict]) -> list[float]:
47     pairs = [[query, d["text"]] for d in docs]
48     raw_scores = reranker.compute_score(pairs)
49     return [sigmoid(s) for s in raw_scores]

```

Listing 3.6: Khởi tạo LLM theo Factory Pattern và Reranker Sigmoid

3.6 BƯỚC 7: Định nghĩa Agent State

Để đảm bảo không trộn lẫn các đoạn trích thô với bằng chứng đã được kiểm duyệt, AgentState phân định rõ từng trường dữ liệu:

```

1 from typing import TypedDict
2
3 class AgentState(TypedDict, total=False):
4     client_id: str
5     session_id: str
6     query: str
7     as_of_date: str
8     memory_context: str
9     route: str # 'rag', 'database', 'general'
10
11     dense_candidates: list[dict]
12     bm25_candidates: list[dict]
13     candidates: list[dict] # Sau RRF và Top-K Rerank
14     relevance_scores: list[float] # Điểm sigmoid từ Reranker
15     crag_action: str # 'CORRECT', 'AMBIGUOUS', 'INCORRECT'
16
17     internal_evidence: list[dict] # Sau Knowledge Refinement
18     external_evidence: list[dict] # Sau Web Search + Selection
19     evidence: list[dict] # Danh sách evidence thống nhất
20
21     generation: dict # Kết quả sinh từ LLM
22     citation_report: dict # Kết quả kiểm tra trình xác citation
23     trace_meta: dict

```

Listing 3.7: Định nghĩa trạng thái tác tử AgentState trong state.py

3.7 BƯỚC 8: Ingestion Pipeline & Hybrid Retrieval

Dữ liệu được nạp đồng thời vào Chroma (Vector index) và rank-bm25 (Lexical index):

```

1 from langchain_chroma import Chroma
2 from rank_bm25 import BM25Okapi
3
4 # 1. Dense Vector Store
5 vectorstore = Chroma.from_texts(
6     texts=[chunk["text"] for chunk in chunks],
7     embedding=embeddings,
8     metadatas=[chunk["metadata"] for chunk in chunks],
9     persist_directory="./chroma",
10    collection_name="legal_corpus_v1",
11 )
12
13 # 2. BM25 Lexical Store
14 def tokenize(text: str) -> list[str]:
15     return text.lower().replace(" ", " ").replace(".", " ").split()
16
17 bm25 = BM25Okapi([tokenize(chunk["text"]) for chunk in chunks])

```

Listing 3.8: Xây dựng Dense Index và BM25 Index

3.8 BƯỚC 9: Xây dựng các Tools

3.8.1 Reciprocal Rank Fusion (RRF)

Hợp nhất kết quả từ Dense và Lexical retriever theo công thức:

$$S_{\text{RRF}}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)} \quad (\text{với } k = 60) \quad (3.1)$$

```

1 def rrf_merge(result_lists: list[list[dict]], k: int = 60) -> list[dict]:
2     scores = {}
3     items = {}
4     for results in result_lists:
5         for rank, item in enumerate(results, start=1):
6             key = item["evidence_id"]
7             scores[key] = scores.get(key, 0.0) + 1.0 / (k + rank)
8             items[key] = item
9     ordered_keys = sorted(scores, key=scores.get, reverse=True)
10    return [items[k] for k in ordered_keys]

```

Listing 3.9: Thuật toán Reciprocal Rank Fusion

3.8.2 Controlled Web Search Tool

```

1 OFFICIAL_DOMAINS = {
2     "vbpl.vn", "vanban.chinhphu.vn", "moj.gov.vn",
3     "chinhphu.vn", "thuvienphapluat.vn"
4 }
5
6 def is_allowed_source(url: str) -> bool:
7     try:
8         host = url.lower().split("/")[2]
9         return any(host == d or host.endswith("." + d) for d in
10                    OFFICIAL_DOMAINS)
11     except Exception:
12         return False

```

Listing 3.10: Bộ lọc tên miền chính thống cho công cụ tìm kiếm

3.9 BƯỚC 10: Xây dựng CRAG Retrieval Evaluator

Điểm số tối đa của tập tài liệu được dùng để quyết định hành động:

```

1 from config import T_LOW, T_HIGH
2
3 def decide_crag_action(scores: list[float]) -> str:
4     best_score = max(scores) if scores else 0.0
5     if best_score >= T_HIGH:
6         return "CORRECT"
7     elif best_score <= T_LOW:
8         return "INCORRECT"
9     else:
10        return "AMBIGUOUS"

```

Listing 3.11: Logic quyết định hành động CRAG dựa trên hai ngưỡng

3.10 BƯỚC 11 – 14: Tinh chế Tri thức, Viết lại Truy vấn & Hợp nhất Bằng chứng

Nguyên tắc Tinh lọc Bằng chứng (Knowledge Refinement)

Khi nhánh là **CORRECT** hoặc **AMBIGUOUS**, hệ thống không đưa nguyên đoạn văn bản dài vào Generator. Mỗi passage được phân rã thành các *legal strips* (từng Khoản/Điểm cụ thể), chấm lại điểm relevance bằng Reranker và chỉ giữ lại các strip đạt ngưỡng điểm tối thiểu.

```

1 def refine_internal(query: str, candidates: list[dict]) -> list[dict]:
2     strips = []
3     for doc in candidates:

```

```

4         for strip in split_into_legal_strips(doc):
5             strips.append(**strip, "parent_id": doc["evidence_id"])
6         ranked = rerank(query, strips)
7         return [s for s in ranked[:8] if s["score"] >= INTERNAL_STRIP_MIN]
8
9     def merge_evidence(internal: list[dict], external: list[dict]) -> list[dict]:
10         all_items = internal + external
11         unique_items = deduplicate_by_locator(all_items)
12         # Ưu tiên: Nguồn pháp lý chính thống > Nguồn báo trí; sau đó xét điểm score
13         return sorted(
14             unique_items,
15             key=lambda x: (x.get("source_priority", 0), x.get("score", 0.0)),
16             reverse=True
17         )[:10]

```

Listing 3.12: Bóc tách Legal Strips và hợp nhất bằng chứng đa nguồn

3.11 BƯỚC 15: Kiểm định Trích dẫn (Citation Validator)

Hệ thống triển khai cơ chế kiểm tra trích dẫn xác định (*Deterministic Citation Validation*):

```

1     def validate_citations(answer: dict, evidence_map: dict, as_of_date: str =
2         None) -> dict:
3         errors = []
4         for claim in answer.get("claims", []):
5             for sid in claim.get("source_ids", []):
6                 if sid not in evidence_map:
7                     errors.append(f"Ồ!Li: source_id không tồn tại trong tập bản
8                         vực: {sid}")
9                     continue
10                if as_of_date and not is_effective_at(evidence_map[sid],
11                    as_of_date):
12                    errors.append(f"Ồ!Li: Bản {sid} không còn hiệu lực tại
13                        ngày {as_of_date}")
14            return {"ok": len(errors) == 0, "errors": errors}

```

Listing 3.13: Hàm kiểm định trích dẫn xác định

3.12 BƯỚC 16: Prompt Sinh câu trả lời có Cấu trúc

```

1  A
2  BN LÀ QTR LÝ ỒN QTR TRA ỨCU TUÂN ỦTH PHÁP LÝ DOANH ỆNGHIP.ỉ
3  Ch ảtr ờli ựa duy ấnh t trên EVIDENCE PHÁP LÝ ỆHIN ẶTI được cung ấcp ướdi đây.
4
5  YÊU ẦCU ẮBT ỘBUC:
6  1. ệTuyt đối không ựt suy đoán ặhoc sáng ạo quy định pháp ậlut.
7  2. ộMi ếkt ậlun pháp lý ậphi được ắgn kèm [source_id] ủa evidence ươtng ứng.
8  3. Không ựt ạo ốs ệhiu ắvn ắbn, tên Ềđiêu/ắKhon ngoài danh ựmc evidence.

```

```

9  4. ếNú evidence có mâu ấthun, ịch rõ ựs khác ệbit và ưu tiên ăvn ảbn có ệhiu
    ựlc.
10 5. ếNú ưcha đũa cn ức, ấtr ờli rõ ràng: "ưCha đũa cn ức pháp lý đểể kt ậlun."
11 6. Định ạngng đầu ra: ấBt ộbuc ấtr ềv đúng JSON Schema quy định:
12 {
13     "answer": "ộNi dung ấtr ờli súc tích...",
14     "claims": [
15         {"text": "ệMnh đểể kt ậlun 1", "source_ids": ["E1", "E2"]},
16         {"text": "ệMnh đểể kt ậlun 2", "source_ids": ["E3"]}
17     ],
18     "abstain": false
19 }
20
21 EVIDENCE:
22 {context}
23
24 CÂU ỎHI:
25 {query}

```

Listing 3.14: Prompt sinh câu trả lời có cấu trúc nghiêm ngặt

3.13 BƯỚC 17: Xây dựng LangGraph Hoàn chỉnh

```

1  from langgraph.graph import StateGraph, END
2
3  def build_crag_graph():
4      g = StateGraph(AgentState)
5
6      # Dang ky cac node
7      g.add_node("router", route_question)
8      g.add_node("db", query_db)
9      g.add_node("retrieve", hybrid_retrieve)
10     g.add_node("evaluate", evaluate_retrieval)
11     g.add_node("refine", refine_internal_node)
12     g.add_node("rewrite", rewrite_query_node)
13     g.add_node("web", web_search_node)
14     g.add_node("select_web", select_external_node)
15     g.add_node("merge", merge_evidence_node)
16     g.add_node("generate", generate_answer)
17     g.add_node("cite_validate", validate_citations_node)
18
19     # Canh dieu kien sau evaluate: CORRECT -> refine, AMBIGUOUS -> refine,
20     # INCORRECT -> rewrite
21     g.add_conditional_edges(
22         "evaluate",
23         lambda s: s["crag_action"],
24         {"CORRECT": "refine", "AMBIGUOUS": "refine", "INCORRECT": "rewrite"}
25     )

```

```
25
26 # Điều hướng sau refine
27 g.add_conditional_edges(
28     "refine",
29     lambda s: "generate" if s["crag_action"] == "CORRECT" else "rewrite",
30     {"generate": "generate", "rewrite": "rewrite"}
31 )
32
33 # Nhanh ngoài
34 g.add_edge("rewrite", "web")
35 g.add_edge("web", "select_web")
36 g.add_conditional_edges(
37     "select_web",
38     lambda s: "merge" if s["crag_action"] == "AMBIGUOUS" else "generate",
39     {"merge": "merge", "generate": "generate"}
40 )
41
42 g.add_edge("merge", "generate")
43 g.add_edge("generate", "cite_validate")
44 g.add_edge("cite_validate", END)
45
46 return g.compile()
```

Listing 3.15: Thiết lập đồ thị LangGraph với 3 nhánh rẽ điều kiện

3.14 BƯỚC 18: Kịch bản Kiểm thử Bắt buộc khi Demo

Table 3.2: 5 Kịch bản kiểm thử bắt buộc cần demo để minh chứng hoạt động của CRAG

Test	Tình huống kiểm thử	Luồng kỳ vọng của hệ thống
Test 1	<i>Correct Case:</i> Câu hỏi in-corpus có đầy đủ căn cứ trong dữ liệu nội bộ.	Điểm số $\geq T_{high}$ $\rightarrow$ CORRECT $\rightarrow$ Refine $\rightarrow$ Generate $\rightarrow$ Citation Validate.
Test 2	<i>Incorrect Case:</i> Câu hỏi về nghị định mới ban hành chưa có trong corpus.	Điểm số $\leq T_{low}$ $\rightarrow$ INCORRECT $\rightarrow$ Rewrite $\rightarrow$ Web Search $\rightarrow$ Select $\rightarrow$ Generate.
Test 3	<i>Ambiguous Case:</i> Corpus có quy định chung nhưng thiếu thủ tục chi tiết.	$T_{low} < \text{Score} < T_{high}$ $\rightarrow$ AMBIGUOUS $\rightarrow$ Refine + Web $\rightarrow$ Merge Evidence $\rightarrow$ Generate.
Test 4	<i>Database Tool:</i> “Văn bản số 01/2021/NĐ-CP còn hiệu lực không?”	Router chuyển sang DB Tool $\rightarrow$ Kiểm tra bảng legal_documents $\rightarrow$ Trả về ngày hiệu lực và trạng thái chính xác.
Test 5	<i>Citation Failure:</i> LLM cố tình trích dẫn nguồn không có thực.	Citation Validator phát hiện vi phạm, chặn câu trả lời và báo lỗi hoặc kích hoạt cơ chế sửa đổi có kiểm soát.

Thiết kế Thực nghiệm & Đánh giá Hệ thống

4.1 Thiết kế Hệ thống Baseline và Nghiên cứu Ablation

Để chứng minh tính vượt trội của Corrective RAG ở chuẩn mực nghiên cứu cao học, không thể chỉ so sánh đơn thuần giữa Traditional RAG và Full CRAG. Cần xây dựng các hệ thống đối chứng để phân tách rạch ròi: *Sự cải thiện đến từ cơ chế hiệu chỉnh tri thức hay chỉ đơn thuần do hệ thống được cấp thêm nguồn Web bên ngoài?*

Table 4.1: Ma trận các hệ thống Baseline thực nghiệm (S0 – S4)

Hệ thống	Evaluator	Web Search	Correction	Mục đích nghiên cứu
S0	Không	Không	Không	LLM-only: Đo lường tri thức nội tại của mô hình, làm mốc tham chiếu không retrieval.
S1	Không	Không	Không	Traditional RAG: Truy hồi Top-K cố định đưa thẳng vào Generator.
S2	Không	Luôn luôn	Không	RAG + Always Web: Luôn tìm kiếm web để tách riêng hiệu ứng “có thêm nguồn ngoài”.
S3	Có	Không	Một phần	RAG + Evaluator: Đánh giá bằng chứng nhưng chỉ quyết định trả lời hoặc từ chối (<i>abstain</i>).
S4	Có	Có điều kiện	Toàn diện	Full CRAG đề xuất: Đầy đủ 3 nhánh Correct, Ambiguous và Incorrect.

Các Biến thể Ablation Study Khuyến nghị

- **Ablation 1 (No Refinement):** Full CRAG loại bỏ bước Knowledge Refinement (đưa nguyên chunk dài vào Generator).
- **Ablation 2 (No Rewrite):** Full CRAG loại bỏ Query Rewrite (dùng trực tiếp câu hỏi thô để search web).
- **Ablation 3 (No Web Selection):** Lấy nguyên trang web fetch được mà không qua bộ lọc relevance strip.
- **Ablation 4 (Dense-only):** Thay thế Hybrid Retrieval bằng truy hồi vector thuần túy BGE-M3.

- **Ablation 5 (No Reranker):** Bỏ mô hình Cross-Encoder, dùng trực tiếp similarity score của vector để routing.

4.2 Thiết kế Tập Calibration và Held-Out Test Set

Quy tắc vàng trong đánh giá học thuật

Tuyệt đối không sử dụng cùng một tập dữ liệu vừa để tìm ngưỡng T_{low}/T_{high} vừa để báo cáo kết quả cuối cùng. Hệ thống phải tách biệt nghiêm ngặt hai tập dữ liệu độc lập:

Table 4.2: Phân bổ các nhóm câu hỏi trong Held-out Test Set (40 câu)

Nhóm câu hỏi kiểm thử	Số lượng	Mục tiêu kiểm chứng
Correct / In-corpus	12 câu	Bằng chứng nội bộ đầy đủ, kiểm tra hiệu quả của Knowledge Refinement.
Incorrect / Out-of-corpus	10 câu	Corpus nội bộ thiếu hụt, kiểm tra khả năng kích hoạt Web Search chính thống.
Ambiguous / Partial evidence	8 câu	Kiểm tra cơ chế hợp nhất tri thức nội bộ và tri thức mở rộng bên ngoài.
Metadata / Database queries	5 câu	Đo lường độ chính xác của Router và khả năng truy vấn dữ liệu có cấu trúc.
Temporal / Conflicting sources	5 câu	Kiểm tra khả năng phát hiện văn bản hết hiệu lực và giải quyết xung đột điều luật.
Tổng cộng	40 câu	Tập dữ liệu độc lập hoàn toàn, khóa trước khi đo benchmark.

Mỗi mẫu trong tập kiểm thử được lưu trữ dưới định dạng JSON chuẩn hóa:

```

1 {
2   "question": "Quy định về thời gian nghỉ việc tối đa đối với vị trí quản lý
3     viên doanh nghiệp?",
4   "type": "ambiguous",
5   "expected_action": "AMBIGUOUS",
6   "as_of_date": "2026-01-01",
7   "expected_source_ids": ["BLLD2019_D25", "VBPL_ND145"],
8   "expected_provisions": ["Điều 25 Bộ luật Lao động 2019"],
9   "ground_truth": "Thời gian nghỉ việc không quá 180 ngày đối với công việc
    của người quản lý doanh nghiệp theo quy định của Bộ luật Doanh nghiệp..."
}
```

Listing 4.1: Cấu trúc của một mẫu dữ liệu trong Test Set

4.3 Hệ thống Chỉ số Đánh giá Toàn diện

4.3.1 Chỉ số Đánh giá Truy hồi (Retrieval Metrics)

- **Recall@K:** Tỷ lệ bằng chứng pháp lý đúng xuất hiện trong Top-K tài liệu truy hồi.
- **MRR (Mean Reciprocal Rank):** Vị trí nghịch đảo trung bình của bằng chứng đúng đầu tiên.
- **nDCG@K (Normalized Discounted Cumulative Gain):** Đánh giá chất lượng của toàn bộ thứ hạng sắp xếp.

4.3.2 Chỉ số Đánh giá Bộ điều hướng & Fallback (Routing & Fallback Metrics)

Nhằm đánh giá chính xác bộ điều hướng có kích hoạt Web Search đúng lúc hay lạm dụng quá mức, chỉ số Fallback được phân tách thành 3 đại lượng:

$$\text{Fallback Precision} = \frac{\text{Số lần kích hoạt Web đúng khi ngoài corpus}}{\text{Tổng số lần hệ thống kích hoạt Web Search}} \quad (4.1)$$

$$\text{Fallback Recall} = \frac{\text{Số câu hỏi ngoài corpus được phát hiện thành công}}{\text{Tổng số câu hỏi ngoài corpus trong tập test}} \quad (4.2)$$

$$\text{False Fallback Rate} = \frac{\text{Số câu hỏi trong corpus nhưng bị gọi nhầm ra Web}}{\text{Tổng số câu hỏi có sẵn trong corpus}} \quad (4.3)$$

4.3.3 Chỉ số Đánh giá Thế hệ Sinh & Độ chuẩn xác Pháp lý (Legal Metrics)

- **RAGAS Metrics:** *Faithfulness* (tính trung thực so với bằng chứng), *Answer Relevancy* (mức độ bám sát câu hỏi), *Context Precision* và *Context Recall*.
- **Citation Accuracy:** Tỷ lệ các trích dẫn pháp lý có source_id tồn tại, khớp chính xác số Điều/Khoản trong văn bản và URL kiểm chứng được.
- **Citation Coverage:** Tỷ lệ các mệnh đề kết luận pháp lý trong câu trả lời có ít nhất một trích dẫn hợp lệ tương ứng.
- **Legal Temporal Correctness:** Đo lường tỷ lệ các văn bản làm căn cứ trả lời còn nguyên giá trị hiệu lực tại thời điểm as_of_date.

4.4 Thí nghiệm Lựa chọn Cặp Ngưỡng Relevance (T_{low} , T_{high})

Trên tập Calibration Set (20 câu), tiến hành quét lưới (*grid search*) các cặp giá trị (T_{low} , T_{high}) với điều kiện tiên quyết $T_{low} < T_{high}$ để chọn điểm cân bằng tối ưu giữa F1-Score và hạn chế False Fallback:

Cam kết trung thực học thuật

Khung bảng trên là mẫu cấu trúc thí nghiệm. Điểm số chỉ được điền sau khi nhóm thực hiện chạy script đánh giá tự động trên tập dữ liệu thực tế. Tuyệt đối không tự điền số liệu giả.

Table 4.3: Bảng khung thực nghiệm quét lưới lựa chọn ngưỡng (Grid Search Calibration)

T_{low}	T_{high}	Correct %	Ambiguous %	Incorrect %	Macro-F1	False Fallback
0.20	0.60	...	...	...	...	...
0.25	0.65	...	...	...	...	...
0.30	0.70	...	...	...	...	...
0.35	0.75	...	...	...	...	...
0.40	0.80	...	...	...	...	...

Mở rộng: Next.js Frontend & Quản lý Session Memory

5.1 Kiến trúc Tích hợp Next.js + FastAPI + Memory

Hệ thống bổ sung giao diện người dùng hiện đại và cơ chế quản lý ngữ cảnh phiên hội thoại sau khi lỗi CRAG đã được kiểm chứng độc lập.

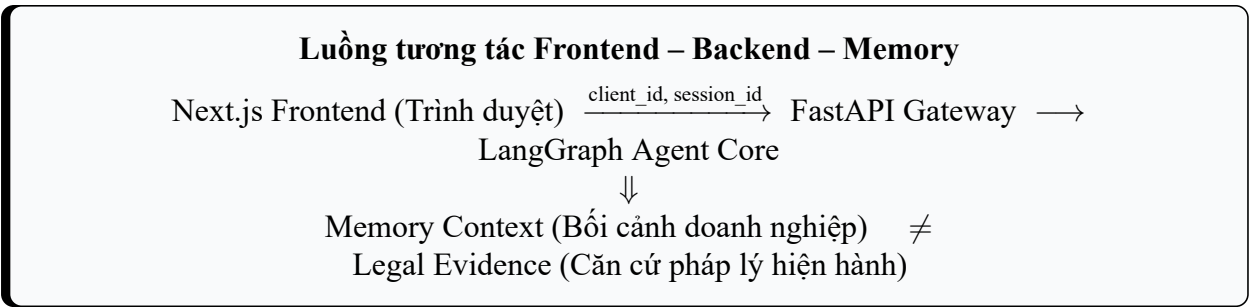


Figure 5.1: Kiến trúc tách biệt giữa ngữ cảnh người dùng và căn cứ pháp lý.

Next.js lưu trữ một mã định danh ẩn danh (`client_id`) trên trình duyệt qua `localStorage` và tự động sinh `session_id` ngẫu nhiên cho từng phiên làm việc. FastAPI tiếp nhận yêu cầu, nạp lịch sử tương tác gần nhất và thông tin hồ sơ doanh nghiệp đã xác thực, truyền vào `AgentState` nhưng cô lập hoàn toàn khỏi danh mục trích dẫn pháp lý.

5.2 Phân cấp Ba Lớp Memory

Table 5.1: Phân tầng 3 cấp độ Memory trong hệ thống

Loại Memory	Vị trí lưu trữ	Mục đích và Phạm vi sử dụng
Working / Session	AgentState & RAM	Duy trì trạng thái câu hỏi, kết quả truy hỏi và bằng chứng của phiên chat tức thời.
Episodic / History	Bảng <code>query_logs</code> (SQLite)	Lưu vết tuần tự các câu hỏi, câu trả lời, route đã chọn và hành động CRAG.
Semantic / Profile	Bảng <code>client_memories</code>	Ghi nhớ các thuộc tính bền vững của doanh nghiệp (loại hình, địa bàn, ngành nghề).

Nguyên tắc an toàn tối thượng (Memory Guardrail)

Memory chỉ đóng vai trò hỗ trợ bối cảnh và giải nghĩa đại từ. **Mọi phán quyết và kết luận pháp lý đều bắt buộc phải được truy hồi lại từ các văn bản hiện hành trong Legal Database hoặc Web chính thống.** Tuyệt đối không sử dụng câu trả lời lưu trong memory như một căn cứ pháp luật.

5.3 Thiết kế Database cho Client, Session và Memory

```
1 CREATE TABLE clients (  
2     id TEXT PRIMARY KEY,  
3     memory_enabled INTEGER DEFAULT 1,  
4     created_at TEXT DEFAULT CURRENT_TIMESTAMP,  
5     last_seen_at TEXT DEFAULT CURRENT_TIMESTAMP  
6 );  
7  
8 CREATE TABLE chat_sessions (  
9     id TEXT PRIMARY KEY,  
10    client_id TEXT NOT NULL,  
11    started_at TEXT DEFAULT CURRENT_TIMESTAMP,  
12    last_active_at TEXT DEFAULT CURRENT_TIMESTAMP,  
13    FOREIGN KEY(client_id) REFERENCES clients(id)  
14 );  
15  
16 CREATE TABLE query_logs (  
17     id INTEGER PRIMARY KEY AUTOINCREMENT,  
18     client_id TEXT NOT NULL,  
19     session_id TEXT NOT NULL,  
20     question TEXT NOT NULL,  
21     answer TEXT,  
22     route TEXT,  
23     crag_action TEXT,  
24     source_type TEXT,  
25     created_at TEXT DEFAULT CURRENT_TIMESTAMP,  
26     FOREIGN KEY(client_id) REFERENCES clients(id),  
27     FOREIGN KEY(session_id) REFERENCES chat_sessions(id)  
28 );  
29  
30 CREATE TABLE client_memories (  
31     id INTEGER PRIMARY KEY AUTOINCREMENT,  
32     client_id TEXT NOT NULL,  
33     memory_key TEXT NOT NULL,  
34     memory_value TEXT NOT NULL,  
35     confidence REAL DEFAULT 1.0,  
36     updated_at TEXT DEFAULT CURRENT_TIMESTAMP,  
37     UNIQUE(client_id, memory_key),
```

```
38 FOREIGN KEY(client_id) REFERENCES clients(id)
39 );
```

Listing 5.1: Schema quản lý Client, Session và Lịch sử truy vấn

5.4 Kiểm soát Semantic Memory qua Danh mục Cho phép (Allow-List)

Hệ thống áp dụng cơ chế trích xuất có kiểm soát nghiêm ngặt nhằm bảo vệ quyền riêng tư và tránh ô nhiễm bộ nhớ:

```
1 ALLOWED_MEMORY_KEYS = {
2     "business_type",      # Ví dụ: Công ty TNHH, Cổ phần
3     "industry",          # Ngành nghề: Xây dựng, Thương mại điện tử
4     "province",          # Địa bàn: Hà Nội, TP.HCM, Long An
5     "frequent_topic",    # Chủ đề hay quan tâm: Lao động, Hợp đồng
6     "preferred_answer"   # Sở thích trình bày: Ngắn gọn, Chi tiết
7 }
8
9 # Prompt trích xuất thông tin bên vùng tu hồi thoại:
10 # 1. Chỉ trích xuất các thuộc tính nằm trong danh mục cho phép
   ALLOWED_MEMORY_KEYS.
11 # 2. Tuyệt đối không lưu mật khẩu, CCCD/CMND, bí mật kinh doanh.
12 # 3. Gan điểm tin cậy (confidence) từ 0.0 đến 1.0.
13 # 4. Trả về mảng JSON rỗng [] nếu không có thông tin phù hợp.
```

Listing 5.2: Cơ chế Allow-List và trích xuất Semantic Memory

5.5 Prompt Trả lời Phân tách Bối cảnh và Căn cứ Pháp lý

```
1 BOI CANH CLIENT (Chỉ dùng để hiệu hoán cảnh, không có giá trị pháp lý):
2 {memory_context}
3
4 EVIDENCE PHÁP LÝ HIỆN TẠI (Nguồn căn cứ duy nhất để đưa ra kết luận):
5 {legal_context}
6
7 CAU HOI HIEN TAI:
8 {query}
9
10 QUY TAC THUC THI:
11 1. Memory chỉ giúp giải thích đại tu (ví dụ: "công ty tôi" -> Công ty TNHH
   tại Long An).
12 2. Tuyệt đối không lấy thông tin từ câu trả lời cũ thay thế cho văn bản pháp
   luật hiện hành.
13 3. Mọi kết luận pháp lý đưa ra bắt buộc phải trích dẫn source_id từ danh mục
   EVIDENCE.
```

- 14 4. Nếu EVIDENCE không có điều khoản tương ứng, nói rõ: "Chưa đủ căn cứ pháp lý để kết luận".

Listing 5.3: Prompt trả lời kết hợp Memory có kiểm soát

5.6 Kịch bản Demo Tái sử dụng Ngữ cảnh trên Trình duyệt

Table 5.2: Kịch bản minh chứng khả năng duy trì ngữ cảnh trên cùng trình duyệt

Thời điểm	Hành động người dùng	Cơ chế xử lý của hệ thống Agent
Lần 1	Truy cập lần đầu qua trình duyệt Next.js.	Ứng dụng tạo <code>client_id</code> ẩn danh lưu vào <code>localStorage</code> và sinh <code>session_id</code> mới.
Lần 1	<i>"Tôi phụ trách doanh nghiệp TNHH tại Long An, muốn hỏi về hợp đồng thử việc."</i>	Hệ thống trích xuất và lưu vào <code>client_memories: business_type = TNHH, province = Long An</code> .
Lần 2	Đóng tab, sau đó mở lại ứng dụng trên cùng trình duyệt.	Next.js tự động đọc lại <code>client_id</code> , sinh <code>session_id</code> mới; backend tự động nạp hồ sơ đã lưu.
Lần 2	<i>"Vậy trường hợp công ty tôi có được ký phụ lục gia hạn thời gian thử việc không?"</i>	Agent nhận diện đại từ <i>"công ty tôi"</i> là Công ty TNHH, sau đó kích hoạt CRAG truy hỏi quy định Bộ luật Lao động 2019 để trả lời chính xác.

5.7 Bảo mật và Tiêu chuẩn Cách ly Dữ liệu (Isolation)

Chỉ số An toàn Tuyệt đối: Hệ thống đặt tiêu chuẩn **Cross-client Contamination = 0**. Trong toàn bộ các ca kiểm thử bảo mật, dữ liệu lịch sử hoặc memory của Client A tuyệt đối không bao giờ được xuất hiện trong phiên truy vấn của Client B. Mọi câu lệnh truy vấn đều bắt buộc có ràng buộc tham số:

WHERE `client_id` = ? AND `session_id` = ?

5.8 Đánh giá Thử nghiệm A/B Memory

Tiến hành đánh giá A/B trên 3 cấu hình thử nghiệm:

- **Cấu hình A (No Memory):** Tác tử xử lý độc lập từng câu hỏi thô.
- **Cấu hình B (Recent History):** Nạp 6 lượt tương tác gần nhất trong phiên.
- **Cấu hình C (History + Semantic Memory):** Kết hợp toàn diện lịch sử và thuộc tính doanh nghiệp.

Hệ thống đo lường độ chính xác giải tham chiếu (*Context Reuse Accuracy*), độ trung thực (*Faithfulness*) và chi phí trễ (*Latency overhead*) để bảo đảm việc bổ sung Memory không làm suy giảm độ chính xác pháp lý của lõi CRAG.

Kế hoạch Thực hiện, Checklist & Hướng dẫn Báo cáo

6.1 Checklist CRAG Core Trước khi Triển khai Mở rộng

14 Tiêu chí Bắt buộc Hoàn thành cho Lỗi CRAG

- [1] Corpus pháp luật lấy từ nguồn chính thức của cơ quan nhà nước, có đầy đủ metadata hiệu lực và URL.
- [2] Phân tách đoạn theo cấu trúc pháp lý (*Legal-aware chunking*: Chương → Điều → Khoản → Điểm).
- [3] Hệ thống Hybrid Retrieval (Dense BGE-M3 + Lexical BM25 + RRF + Reranker) hoạt động ổn định.
- [4] Retrieval Evaluator phân loại độc lập và chính xác 3 nhánh: Correct, Ambiguous, Incorrect.
- [5] Hai ngưỡng T_{low} và T_{high} được tối ưu bằng thực nghiệm quét lưới trên Calibration Set.
- [6] Nhánh **Correct** kích hoạt thành công cơ chế Knowledge Refinement bóc tách legal strips.
- [7] Nhánh **Incorrect** kích hoạt Query Rewrite → Controlled Web Search → Evidence Selection.
- [8] Nhánh **Ambiguous** kích hoạt đồng thời Refinement và Web Search → Merge Evidence đa nguồn.
- [9] Generator chỉ được quyền sử dụng các đoạn trích đã vượt qua bộ lọc bằng chứng.
- [10] Citation Validator phát hiện và chặn đứng mọi trích dẫn sai số hiệu hoặc không có thật.
- [11] Sở hữu tập kiểm thử độc lập Held-out Test Set (40 câu) có ground-truth chi tiết.
- [12] Chạy thực nghiệm đo lường đầy đủ trên các hệ thống Baseline: S0, S1, S2, S3 và S4 (Full CRAG).
- [13] Phân tích ít nhất một trường hợp thất bại thực tế (*Failure Case Analysis*) của hệ thống.
- [14] Cam kết không sử dụng bất kỳ số liệu giả định hay suy đoán nào trong báo cáo.

6.2 Lộ trình 17 Giai đoạn Triển khai Dự án

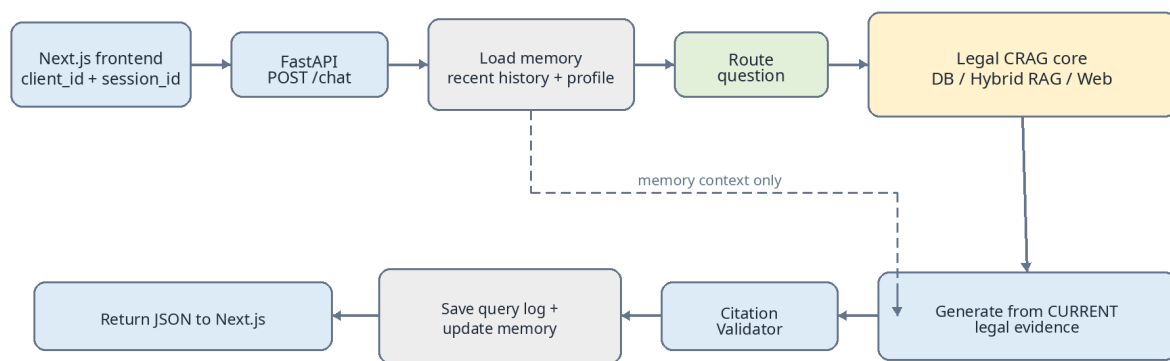
6.3 Đề cương Chi tiết 8 Chương của Báo cáo Kỹ thuật

Báo cáo kỹ thuật tốt nghiệp hoặc đồ án môn học chuẩn Thạc sĩ được phân bổ theo 8 chương:

1. **Chương 1 – Giới thiệu:** Bối cảnh ảo giác do retrieval sai trong lĩnh vực luật; bài toán trợ lý

Table 6.1: Kế hoạch thực hiện dự án theo 17 giai đoạn chuẩn hóa

Giai đoạn	Công việc chi tiết	Sản phẩm đầu ra (Deliverables)
1	Xác định phạm vi và chính sách nguồn dữ liệu	Danh mục 50–100 văn bản pháp lý chính thống.
2	Thu thập và bóc tách văn bản thô	Thư mục <code>data/raw</code> và <code>data/processed</code> .
3	Chuẩn hóa metadata và quan hệ sửa đổi/bổ sung	SQLite database <code>app.db</code> với bảng quan hệ.
4	Xây dựng Calibration Set và Held-out Test Set	<code>calibration_set.json</code> và <code>test_set.json</code> .
5	Cài đặt baseline truy hồi Dense Vector	Vector Store Chroma với embedding BGE-M3.
6	Bổ sung BM25, thuật toán RRF và Cross-Encoder	Module Hybrid Retrieval hoàn chỉnh.
7	Cài đặt Retrieval Evaluator và chọn ngưỡng	Cặp giá trị tối ưu (T_{low}, T_{high}) trên Calibration Set.
8	Hiện thực cơ chế Knowledge Refinement	Nhánh CORRECT bóc tách legal strips.
9	Viết lại truy vấn và tìm kiếm web chính thống	Nhánh INCORRECT với bộ lọc allow-list.
10	Thuật toán hợp nhất và khử trùng lặp bằng chứng	Nhánh AMBIGUOUS tích hợp đa nguồn.
11	Xây dựng Citation Validator xác định	Module kiểm tra tính xác thực của citation.
12	Hoàn thiện LangGraph Agent Core	StateGraph điều phối toàn bộ workflow CRAG.
13	Đo lường Baseline, Ablation và chỉ số RAGAS	Bảng dữ liệu thực nghiệm và vết LangFuse.
14	Phát triển FastAPI Gateway và Next.js Frontend	Giao diện tra cứu và bảng trích dẫn citation.
15	Tích hợp Recent History và Semantic Memory	Module lưu trữ phiên hội thoại SQLite.
16	Đánh giá thử nghiệm A/B Memory	Báo cáo an toàn cách ly Cross-client = 0.
17	Hoàn thiện Báo cáo kỹ thuật và Slide thuyết minh	Bản báo cáo LaTeX và mã nguồn nghiệm thu.



Memory helps interpret the user context; every legal conclusion is re-grounded in current DB/RAG/Web evidence.

Figure 6.1: Thứ tự thực hiện chuẩn mực: Ưu tiên dữ liệu, đo lường và lỗi CRAG trước khi làm UI.

tuân thủ pháp lý doanh nghiệp; mục tiêu và 5 câu hỏi nghiên cứu; phạm vi; các đóng góp chính của đề tài.

2. **Chương 2 – Cơ sở Lý thuyết:** Tổng quan LLM nội bộ; tác tử AI có trạng thái (Stateful AI Agent); kiến trúc RAG cổ điển; Dense Retrieval & BGE-M3; BM25 & Hybrid Search; Reranker Cross-Encoder; LangGraph; nguyên lý Corrective RAG; khung đánh giá RAGAS; lý thuyết về Memory trong Agent.
3. **Chương 3 – Phân tích & Thiết kế Hệ thống:** Đặc tả yêu cầu người dùng; kiến trúc 3 Layer; workflow chi tiết của LangGraph; thiết kế cơ sở dữ liệu pháp lý và quan hệ sửa đổi; mô hình AgentState; chính sách kiểm soát nguồn web; thiết kế hệ thống trích dẫn.
4. **Chương 4 – Chuẩn bị Dữ liệu & Tri thức:** Thu thập dữ liệu từ các cổng công quyền; kỹ thuật Legal-aware chunking; chuẩn hóa metadata; mô hình hóa lịch sử văn bản; lập chỉ mục vector Chroma và chỉ mục từ khóa BM25.
5. **Chương 5 – Xây dựng Hệ thống Agent:** Hiện thực Hybrid Retrieval, RRF và Reranker; xây dựng Evaluator với hai ngưỡng; chi tiết hóa 3 nhánh CRAG; Generator có cấu trúc JSON; Citation Validator; kết nối FastAPI và Next.js UI; giám sát qua LangFuse.
6. **Chương 6 – Thiết kế Thực nghiệm:** Tách biệt Calibration Set và Held-out Test Set; thiết kế các hệ thống Baseline S0–S4; các phép thử Ablation; hệ thống các chỉ số: Retrieval, Routing/-Fallback, Generation (RAGAS), Legal Metrics; quy trình quét lưới chọn ngưỡng.
7. **Chương 7 – Kết quả Thực nghiệm & Thảo luận:** Đánh giá chất lượng truy hồi; ma trận nhầm lẫn (Confusion Matrix) của Evaluator; hiệu quả xử lý 3 nhánh CRAG; phân tích các ca gọi Web Fallback; độ chính xác trích dẫn; phân tích các ca thất bại thực tế (*Failure Cases*).
8. **Chương 8 – Kết luận & Hướng Phát triển:** Đánh giá mức độ hoàn thành so với mục tiêu ban đầu; các hạn chế tồn đọng; đề xuất hướng mở rộng (Knowledge Graph, Human-in-the-loop, quản lý phiên bản tự động).

Phụ lục & Tài liệu Tham khảo

7.1 PHỤ LỤC A: Danh mục File Mã nguồn Chuẩn hóa

7.2 PHỤ LỤC B: Gợi ý Phân công Công việc Nhóm (5 Thành viên)

7.3 PHỤ LỤC C: Tiêu chí Tự Kiểm tra Chất lượng Trước khi Nộp

Bảng kiểm tra điều kiện nghiệm thu

- Có ít nhất một ca kiểm thử **Correct** trả lời chính xác và dẫn nguồn hợp lệ.
- Có ít nhất một ca kiểm thử **Incorrect** kích hoạt thành công tìm kiếm web có kiểm soát.
- Có ít nhất một ca kiểm thử **Ambiguous** hợp nhất thành công tri thức nội bộ và mở rộng.
- Có ít nhất một truy vấn metadata trực tiếp từ cơ sở dữ liệu quan hệ.
- Vết thực thi (*Execution Trace*) thể hiện rõ điểm số đánh giá và nhánh rẽ được chọn.
- Bảng thực nghiệm so sánh đầy đủ giữa Traditional RAG, RAG + Always Web và Full CRAG.
- Tập Calibration Set được phân tách độc lập hoàn toàn khỏi Held-out Test Set.
- Có Citation Validator và chứng minh được khả năng chặn câu trả lời trích dẫn sai.
- Có phân tích ca thất bại thực tế kèm nguyên nhân gốc rễ.
- Tuyệt đối không dùng dữ liệu memory làm căn cứ pháp lý.
- Đạt chỉ số an toàn tuyệt đối: **Cross-client Contamination = 0**.

7.4 PHỤ LỤC D: So sánh Chi tiết các Thay đổi giữa Bản cũ và Bản V3

Tài liệu Tham khảo

- [1] Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). *Corrective Retrieval Augmented Generation*. arXiv preprint arXiv:2401.15884v3.
- [2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ...& Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

Table 7.1: Danh mục các module mã nguồn trong hệ thống Legal CRAG V3

Tên file / Thư mục	Mục đích và Chức năng thực thi
config.py	Cấu hình tham số mô hình, Top-K, ngưỡng (T_{low} , T_{high}) và allow-list nguồn web.
llm.py	Khởi tạo LLM đa Provider (Groq, OpenAI, OpenRouter, Ollama) theo Factory Pattern và Embeddings BGE-M3.
legal/parser.py	Phân tách cấu trúc văn bản pháp luật theo Chương, Điều, Khoản, Điểm.
retrieval/dense.py	Truy hồi vector ngữ nghĩa với ChromaDB.
retrieval/bm25.py	Truy hồi từ khóa chính xác với thuật toán BM25Okapi.
retrieval/fusion.py	Hợp nhất danh sách xếp hạng ứng viên bằng giải thuật Reciprocal Rank Fusion (RRF).
retrieval/reranker.py	Mô hình Cross-Encoder xếp hạng lại và tính relevance score chuẩn hóa Sigmoid.
retrieval/refine.py	Bóc tách và tinh chế các legal strips đạt ngưỡng liên quan tối thiểu.
tools/database.py	Truy vấn dữ liệu có cấu trúc từ SQLite (số hiệu, tình trạng hiệu lực).
tools/web_search.py	Viết lại truy vấn, tìm kiếm web và lọc tên miền chính thống cho phép.
legal/citations.py	Kiểm tra tính xác thực của source_id, số Điều/Khoản và URL trích dẫn.
agent/state.py	Định nghĩa cấu trúc trạng thái AgentState dạng TypedDict.
agent/nodes.py	Cài đặt các hàm xử lý logic tại từng node của đồ thị LangGraph.
agent/graph.py	Thiết lập đồ thị StateGraph và các cạnh điều kiện rẽ 3 nhánh CRAG.
memory/store.py	Quản lý cơ sở dữ liệu lịch sử phiên, nhật ký truy vấn và thuộc tính client.
memory/extractor.py	Trích xuất thông tin doanh nghiệp bền vững dựa trên danh mục cho phép.
eval/metrics.py	Tính toán các chỉ số Retrieval, Routing Fallback, RAGAS và Citation.
eval/run_eval.py	Tự động chạy đánh giá hàng loạt trên Calibration Set và Held-out Test Set.
api/main.py	Hạ tầng FastAPI cung cấp các API REST/JSON phục vụ frontend.
frontend/	Ứng dụng Next.js + TypeScript cung cấp giao diện tương tác người dùng.

Table 7.2: Mô hình phân công trách nhiệm trong nhóm thực hiện đề tài

Vai trò thành viên	Nội dung công việc phụ trách chính
Thành viên 1 – Data Engineer	Thu thập văn bản từ cổng thông tin nhà nước; viết parser phân tách điều khoản; chuẩn hóa metadata và quan hệ sửa đổi; thiết kế và quản trị SQLite database.
Thành viên 2 – Retrieval Specialist	Triển khai Chroma vector store; cài đặt rank-bm25; lập trình giải thuật hợp nhất RRF; tích hợp BGE-Reranker; đo lường các chỉ số Recall@K và MRR.
Thành viên 3 – Agent/Crag Core	Hiện thực bộ đánh giá Retrieval Evaluator; cài đặt 3 nhánh Correct, Ambiguous, Incorrect; xây dựng Knowledge Refinement và Web Search; lập trình LangGraph.
Thành viên 4 – Evaluation & QA	Xây dựng ground-truth cho Calibration Set và Held-out Test Set; thực hiện quét lưới chọn ngưỡng; đo lường RAGAS, Fallback metrics; cấu hình LangFuse trace.
Thành viên 5 – Frontend & Memory	Phát triển ứng dụng Next.js và API FastAPI; xây dựng hệ thống Session Memory và Semantic Profile; thực hiện kiểm thử an toàn cách ly dữ liệu Cross-client = 0.

[3] Asai, A., Sewon, M., Jiang, Z., Chen, X., Zettlemoyer, L., & Hajishirzi, H. (2024). *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*. International Conference on Learning Representations (ICLR).

[4] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv preprint arXiv:2309.15217.

[5] Quốc hội Nước Cộng hòa Xã hội Chủ nghĩa Việt Nam. (2020). *Luật Doanh nghiệp số 59/2020/QH14*.

[6] Quốc hội Nước Cộng hòa Xã hội Chủ nghĩa Việt Nam. (2019). *Bộ luật Lao động số 45/2019/QH14*.

[7] Đề cương Đồ án Môn học: *Nghiên cứu và Phát triển Tác tử Trí tuệ Nhân tạo Nâng cao (Advanced AI Agents with LangGraph & Ollama)*.

[8] Groq Inc. (2024). *Groq LPU Inference Engine Documentation & API Reference*. <https://groq.com>.

[9] OpenRouter. (2024). *Unified LLM Interface & Routing Documentation*. <https://openrouter.ai>.

Table 7.3: Bảng đối chiếu 15 điểm cải tiến then chốt của Phiên bản V3

Hạng mục	Thiết kế trong các bản cũ	Cải tiến đột phá trong Bản V3
Trọng tâm nghiên cứu	CRAG và Memory phát triển dần trái ngang hàng.	CRAG Core và Thực nghiệm là trọng tâm cốt lõi ; Next.js và Memory là phần mở rộng.
Kiến trúc hệ thống	Router → DB/RAG → Grader → Web/Generate.	Kiến trúc 3 Layer chuẩn mực : Interface & API, Agent & Correction, Knowledge & Data.
Nhánh Ambiguous	Lặp lại truy hồi máy móc với biến MAX_ITER.	Tinh lọc nội bộ + Viết lại truy vấn + Web Search → Hợp nhất bằng chứng đa nguồn .
Cơ chế ngưỡng	Dùng một ngưỡng duy nhất 0.5; thử nghiệm cảm tính.	Hai ngưỡng độc lập (T_{low}, T_{high}) quét lưới tối ưu trên Calibration Set riêng biệt.
Retrieval Evaluator	Dùng prompt yêu cầu chính LLM Generator tự chấm điểm.	Sử dụng mô hình Cross-Encoder chuyên biệt tính relevance score chuẩn hóa hàm Sigmoid.
Cơ chế truy hồi	Truy hồi Dense vector đơn lẻ với Chroma Top-5.	Hybrid Retrieval : Dense BGE-M3 + Lexical BM25 → Hợp nhất RRF → Rerank Top-5.
Phương pháp cắt đoạn	Recursive chunking cố định 700 ký tự / 100 overlap.	Legal-aware chunking theo cấu trúc: Chương → Điều → Khoản → Điểm.
Thiết kế Database	Bảng thô sơ với trường đơn thay_the_boi.	Schema 4 bảng quan hệ : documents, provisions, versions, relations.
Cơ chế Web Fallback	Lấy trực tiếp kết quả DuckDuckGo đưa vào sinh lời.	Viết lại truy vấn → Lọc tên miền công quyền → Tải trang → Bóc tách bằng chứng .
Kiểm định trích dẫn	Yêu cầu LLM tự ghi số Điều/Khoản trong prompt.	Gán locator xác định và Citation Validator kiểm tra chặt chẽ trước khi hiển thị.
Khung đánh giá	Chỉ so sánh RAG vs CRAG; test set tạo muộn.	Ma trận S0-S4, Ablation , phân tách Calibration vs Held-out Test Set từ đầu.
Quản lý Memory	Memory gán cứng theo định danh người dùng chung.	Phân 3 tầng memory , liên kết theo cặp ản danh client_id/session_id cô lập.
Thứ tự thực hiện	DB → RAG → CRAG → LangGraph → Test → UI.	Dữ liệu → Thiết kế Test → Hybrid → Evaluator → CRAG 3 nhánh → UI/Memory .
Triển khai hệ thống	Ôm đồm hạ tầng sản xuất phức tạp.	Tập trung tối đa vào độ chính xác khoa học , tái lập được trong môi trường cục bộ.
Tầng LLM	Cố định sử dụng duy nhất Ollama nội bộ.	Kiến trúc Đa Provider linh hoạt : Hỗ trợ Groq siêu tốc, OpenAI GPT-4o, OpenRouter và Ollama local qua biến cấu hình.